Affine Caesar Cipher

36. Write a C program for Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p, substitute the ciphertext letter C: C = E([a, b], p) = (ap + b) mod 26 A basic requirement of any encryption algorithm is that it be one-to-one. That is, if p q, then E(k, p) E(k, q).Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a. For example, for a = 2 and b = 3, then E([a, b], 0) = E([a, b], 13) = 3.

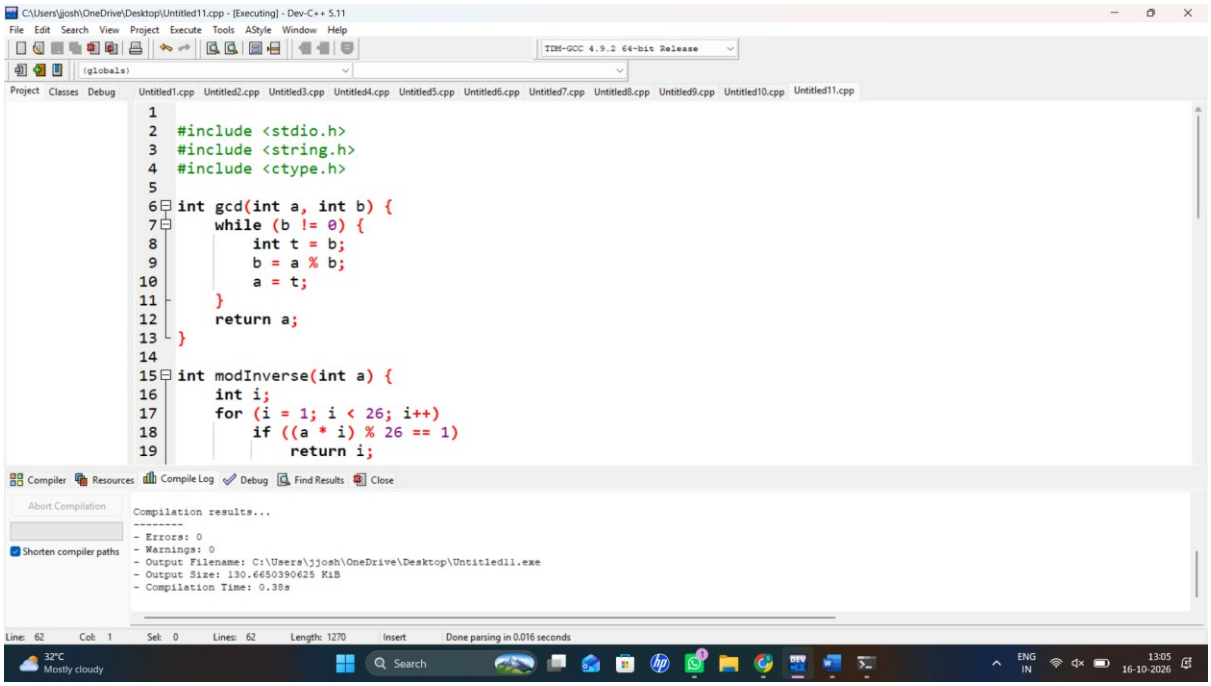
Aim:

To implement the Affine Caesar cipher in C and demonstrate that encryption is one-to-one only when \(a\) and 26 are relatively prime.

Algorithm:

1. Read the plaintext and key values \(a\) and \(b\).
2. Check whether \(\gcd(a,26)=1\).
3. Convert each letter into a number from 0 to 25.
4. Encrypt using \(C=(ap+b)\bmod26\).
5. Find the modular inverse of \(a\) modulo 26.
6. Decrypt using \(p=a^{-1}(C-b)\bmod26\).

Program:



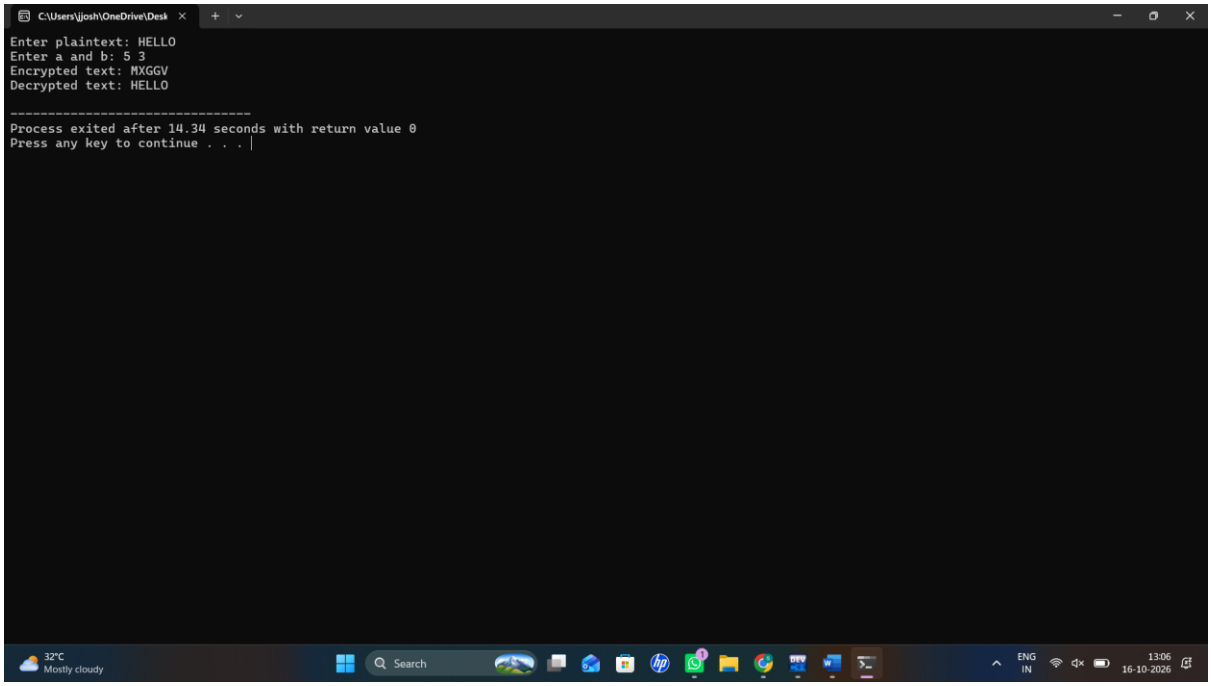
```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int modInverse(int a) {
    int i;
    for (i = 1; i < 26; i++)
        if ((a * i) % 26 == 1)
            return i;
```

```
Compilation results...
--------
- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\jjosh\OneDrive\Desktop\Untitled11.exe
- Output Size: 130.6650390625 KiB
- Compilation Time: 0.38s
```

Input/Output:



```
Enter plaintext: HELLO
Enter a and b: 5 3
Encrypted text: MXGGV
Decrypted text: HELLO

--------------------------------
Process exited after 14.34 seconds with return value 0
Press any key to continue . . .
```

Result:

The Affine Caesar cipher was implemented successfully. A valid key requires \(\gcd(a,26)=1\). For example, \(a=2,b=3\) is invalid because multiple plaintext letters map to the same ciphertext letter, making unique decryption impossible.